

CASOS DE PRUEBA

Sokoban – Versión Académica

Algoritmos y Programación II (APO2)

Fecha	28 de marzo de 2026
Integrantes	[Nombres del equipo]
Versión	1.0

1. Introducción

Este documento especifica los casos de prueba para el modelo del juego Sokoban. El diseño está orientado a validar las reglas del juego de forma aislada de la interfaz gráfica, de modo que cada test sea determinista, repetible y ejecutable en menos de 3 segundos total con JUnit.

Los casos cubren movimiento del jugador, empuje de cajas, detección de colisiones, condición de victoria, y el comportamiento de las estructuras de datos requeridas (Pila, Cola, Tabla Hash, Cola de Prioridad, Lista Enlazada y ABB).

Convención de coordenadas

Todas las posiciones se expresan como (fila, columna), donde (0,0) es la esquina superior izquierda del tablero. Las paredes se representan con '#', espacios libres con '.', el jugador con 'P', cajas con 'B' y metas con 'G'.

2. Casos de Prueba

TC-001 – Movimiento del jugador en espacio libre

ID	TC-001
Objetivo	El jugador debe poder desplazarse en las cuatro direcciones cuando las celdas adyacentes están libres, y el contador de movimientos debe incrementar exactamente una vez por movimiento válido.
Precondición	Tablero inicializado con loadLevel(); no hay cajas ni metas.
Pasos	1. Tablero 7×7 con paredes en el perímetro, jugador en (3,3).2. Llamar moveUp() → jugador esperado en (2,3).3. Llamar moveRight() → jugador esperado en (2,4).4. Llamar moveDown() → jugador esperado en (3,4).5. Llamar moveLeft() → jugador esperado en (3,3).6. Leer getMoveCount().
Resultado esperado	Posición final (3,3); getMoveCount() == 4; ninguna excepción lanzada.

TC-002 – Colisión con pared – el jugador no atraviesa límites

ID	TC-002
Objetivo	Un intento de movimiento cuya celda destino sea pared debe dejar la posición del jugador y todos los contadores sin cambios.
Precondición	Jugador ya fue colocado adyacente a la pared mediante loadLevel().
Pasos	1. Jugador en (1,3), pared en (0,3) (borde superior).2. Llamar moveUp().3. Leer posición y getMoveCount().
Resultado esperado	Jugador permanece en (1,3); getMoveCount() == 0; moveUp() retorna false.

TC-003 – Empuje de caja a celda libre

ID	TC-003
Objetivo	Cuando el jugador empuja una caja hacia un espacio libre, tanto jugador como caja deben desplazarse y los contadores de movimiento y empuje deben incrementar.
Precondición	Verificar con getBoard()[3][4].getType() == FLOOR antes del test.
Pasos	1. Jugador en (3,2), caja en (3,3), espacio libre en (3,4).2. Llamar moveRight().3. Leer posiciones, getMoveCount() y getPushCount().
Resultado esperado	Jugador en (3,3); caja en (3,4); getMoveCount()==1; getPushCount()==1; moveRight() retorna true.

TC-004 – Caja bloqueada por pared – movimiento rechazado

ID	TC-004
Objetivo	Si la celda detrás de la caja es una pared, el sistema no debe permitir el empuje. El estado del tablero debe quedar intacto.
Pasos	1. Jugador en (3,2), caja en (3,3), pared en (3,4).2. Llamar moveRight().3. Verificar estado completo.
Resultado esperado	Jugador en (3,2); caja en (3,3); getMoveCount()==0; getPushCount()==0; moveRight() retorna false.

TC-005 – Dos cajas consecutivas – empuje no permitido

ID	TC-005
Objetivo	El jugador nunca puede empujar dos cajas a la vez. Si el intento se realiza, el sistema debe rechazarlo sin alterar el tablero.
Pasos	1. Jugador en (3,1), caja1 en (3,2), caja2 en (3,3).2. Llamar moveRight().3. Verificar que las tres entidades mantienen sus posiciones.
Resultado esperado	Todas las posiciones sin cambio; contadores en cero; moveRight() retorna false.

TC-006 – Caja empujada sobre meta

ID	TC-006
Objetivo	Al empujar una caja sobre una celda meta, el sistema debe reconocer el estado 'caja sobre meta' y marcarlo visualmente/lógicamente.
Pasos	1. Jugador en (3,2), caja en (3,3), meta en (3,4).2. Llamar moveRight().3. Consultar getBoard()[3][4].getType() y isBoxOnGoal().
Resultado esperado	getBoard()[3][4].getType() == BOX_ON_GOAL; isBoxOnGoal(3,4) == true; getPushCount()==1.
Postcondición	El contador de cajas-sobre-meta debe incrementar en 1.

TC-007 – Condición de victoria – todas las cajas en meta

ID	TC-007
Objetivo	isLevelComplete() debe retornar true únicamente cuando la cantidad de cajas sobre metas es igual al total de cajas del nivel. Las comprobaciones parciales deben retornar false.
Precondición	Nivel diseñado para que las tres metas sean accesibles en el orden descrito.

Pasos	1. Nivel con 3 cajas y 3 metas.2. Empujar caja1 a meta → isLevelComplete() debe ser false.3. Empujar caja2 a meta → isLevelComplete() debe ser false.4. Empujar caja3 a meta → isLevelComplete() debe ser true.5. Verificar que el estado del juego pase a WON.
Resultado esperado	Pasos 2 y 3 retornan false; paso 4 retorna true; gameState() == GameState.WON.

TC-008 – Carga de nivel desde JSON – validación de estructura

ID	TC-008
Objetivo	El parser debe interpretar correctamente un JSON de nivel bien formado y poblar el tablero con los elementos en las posiciones correctas.
Precondición	jsonValido es el string definido en el anexo de datos de prueba.
Pasos	1. Llamar loadLevelFromJson(jsonValido) con el JSON de prueba.2. Verificar dimensiones con getWidth() y getHeight().3. Verificar getPlayerPosition().4. Recorrer board para contar cajas y metas.
Resultado esperado	getWidth()==7; getHeight()==6; jugador en (3,3); 1 caja; 1 meta; paredes en fila 0 y fila 5.

TC-009 – Deshacer movimiento simple (Stack)

ID	TC-009
Objetivo	undo() debe restaurar el estado completo anterior al último movimiento, decrementar getMoveCount() y reducir en 1 el tamaño de la pila de historial.
Pasos	1. Jugador en (3,3); registrar stackSize = getHistoryStack().size().2. Llamar moveRight() → jugador en (3,4); getMoveCount()==1.3. Llamar undo().4. Verificar posición, contador y tamaño de pila.
Resultado esperado	Jugador en (3,3); getMoveCount()==0; getHistoryStack().size()==stackSize; undo() retorna true.

TC-010 – Deshacer empuje de caja (Stack)

ID	TC-010
Objetivo	undo() tras un empuje debe revertir tanto la posición del jugador como la de la caja, y debe decrementar push counter y move counter.
Pasos	1. Jugador en (3,2), caja en (3,3), libre en (3,4).2. Llamar moveRight() → jugador (3,3), caja (3,4).3. Llamar undo().

Resultado esperado	Jugador en (3,2); caja en (3,3); getMoveCount()==0; getPushCount()==0.
--------------------	--

TC-011 – Tabla Hash – detección de estado ya visitado

ID	TC-011
Objetivo	La función de hash debe producir el mismo hash para estados idénticos del tablero, y hasVisited() debe retornar true al encontrar un estado duplicado.
Precondición	Se usa Zobrist hashing o equivalente para garantizar uniformidad.
Pasos	1. Capturar hashA = computeHash(estadoinicial).2. Guardar con markVisited(estadoinicial).3. Hacer 4 movimientos que formen un ciclo y regresen al estado inicial.4. Capturar hashB = computeHash(estadocurrente).5. Llamar hasVisited(estadocurrente).
Resultado esperado	hashA == hashB; hasVisited() retorna true; sin colisiones falsas positivas en el ciclo.

TC-012 – Cola de entrada – procesamiento FIFO (Queue)

ID	TC-012
Objetivo	Los comandos encolados deben ejecutarse en estricto orden FIFO; ningún comando debe perderse ni reordenarse.
Precondición	Jugador en (3,3); tablero abierto sin obstáculos.
Pasos	1. Encolar: UP, RIGHT, DOWN, LEFT.2. Llamar processNextCommand() cuatro veces.3. Verificar posición final y que la cola quede vacía.
Resultado esperado	Los 4 comandos se ejecutaron en ese orden; jugador en posición coherente con la secuencia; getCommandQueue().isEmpty()==true.

TC-013 – Cola de Prioridad – tabla de mejores puntuaciones (top-K)

ID	TC-013
Objetivo	La cola de prioridad debe conservar únicamente las K puntuaciones más bajas (menor cantidad de movimientos). Al insertar más de K elementos, el peor score debe eliminarse automáticamente.
Pasos	1. Configurar K=5.2. Insertar 8 scores: 40,15,62,28,9,51,33,47.3. Llamar getLeaderboard() y verificar contenido y orden.
Resultado esperado	Leaderboard contiene exactamente [9,15,28,33,40] en orden ascendente; scores 47,51,62 fueron descartados; cada inserción O(log K).

TC-014 – Lista Enlazada – inserción, eliminación y acceso por índice

ID	TC-014
Objetivo	La implementación propia de LinkedList debe soportar inserción al final, eliminación por valor y acceso por índice, sin usar ninguna colección de java.util.
Precondición	Confirmar con reflexión o inspección que la clase no declara ningún campo de tipo java.util.List/ArrayList.
Pasos	1. Crear lista vacía.2. Añadir: A, B, C, D, E → verificar size()==5.3. Llamar remove('C') → verificar size()==4.4. Llamar get(2) → debe retornar 'D'.5. Llamar get(0) → debe retornar 'A'.
Resultado esperado	size()==4; get(2)=='D'; get(0)=='A'; no se lanza IndexOutOfBoundsException; sin imports de java.util.*.

TC-015 – Árbol Binario de Búsqueda – inserción y recorrido inorden

ID	TC-015
Objetivo	El ABB debe mantener la propiedad de ordenamiento (izq < raíz < der) y el recorrido inorden debe producir los elementos en orden ascendente.
Pasos	1. Insertar en orden: 50, 30, 70, 20, 40, 60, 80.2. Llamar search(40) y search(25).3. Llamar inOrder() y capturar la secuencia.
Resultado esperado	search(40)==true; search(25)==false; inOrder() retorna [20,30,40,50,60,70,80].

TC-016 – Contadores de movimientos y empujes – exactitud

ID	TC-016
Objetivo	Los contadores deben reflejar únicamente acciones válidas: un movimiento inválido no debe sumar ni a moves ni a pushes.
Pasos	1. Ejecutar 3 movimientos libres.2. Ejecutar 2 empujes válidos de caja.3. Intentar 1 movimiento inválido (contra pared).4. Leer getMoveCount() y getPushCount().
Resultado esperado	getMoveCount()==5; getPushCount()==2; el movimiento inválido no altera ningún contador.

TC-017 – Múltiples undos y reanudación de juego (Stack)

ID	TC-017
Objetivo	Después de deshacer varios movimientos y continuar jugando, la pila y los contadores deben reflejar únicamente el historial efectivo.
Pasos	1. Hacer 5 movimientos → <code>stack.size()==5</code> , <code>getMoveCount()==5</code> . 2. Llamar <code>undo()</code> tres veces → <code>stack.size()==2</code> , <code>getMoveCount()==2</code> . 3. Hacer 2 movimientos nuevos → <code>stack.size()==4</code> , <code>getMoveCount()==4</code> . 4. Verificar que los 3 estados deshechos ya no están en la pila.
Resultado esperado	Tras el paso 3: <code>stack.size()==4</code> ; <code>getMoveCount()==4</code> ; los estados previos al <code>undo</code> no son recuperables.

3. Matriz de Trazabilidad

Requisito / Componente	Casos de Prueba
Movimiento del jugador	TC-001, TC-002
Empuje de cajas	TC-003, TC-004, TC-005, TC-006
Condición de victoria	TC-007
Carga de nivel (JSON)	TC-008
Pila – Undo	TC-009, TC-010, TC-017
Tabla Hash	TC-011
Cola – Input buffer	TC-012
Cola de Prioridad	TC-013
Lista Enlazada	TC-014
Árbol Binario de Búsqueda	TC-015
Estadísticas del juego	TC-016

4. Análisis de Complejidad

Estructura	Inserción	Búsqueda	Eliminación	Espacio
Tabla Hash (transposición)	$O(1)$ prom.	$O(1)$ prom.	$O(1)$ prom.	$O(n)$
Pila (historial undo)	$O(1)$	–	$O(1)$	$O(m)$
Cola (buffer comandos)	$O(1)$	–	$O(1)$	$O(k)$
Cola de Prioridad (top-K)	$O(\log K)$	$O(1)$ mín	$O(\log K)$	$O(K)$
Lista Enlazada	$O(1)$ cabeza	$O(n)$	$O(n)$	$O(n)$
ABB (promedio balanceado)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$

m = número de movimientos; k = comandos en buffer; n = entradas en la estructura; K = tamaño del leaderboard.

